Ref. Ares(2024)6914023 - 30/09/2024

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D7.8: Software tools on public Git repository (second release)

Work Package: WP7

Authors: Ernesto Casablanca and Sadegh Soudjani

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1st October 2022

Duration: 36 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D7.8

Issue: 1.1

Issue Date: 26.09.2024

Prepared by: Max Planck Institute for Software Systems (MPI-SWS)

Approved: Yes

Dissemination Status: Confidential, only for members of the consortium

History Chart:

Issue	Date (DD.MM.YYYY)	Updated Sections	Cause of Change	Implemented by
1.0	09.09.2024	All sections	New Document	Ernesto Casablanca
1.1	25.09.2024	All sections	Revising and Polishing	Sadegh Soudjani

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

List of Figures 3

Acronyms 4

1 Introduction 5
- 1.1 SymAware: High-level Description of the Project . . . 5
- 1.2 Purpose and Contents of This Report . . . 7

2 Git repository 8
- 2.1 Repository . . . 8
- 2.2 Installation . . . 9

3 Examples 11
- 3.1 Simple example . . . 11
- 3.2 Communication example . . . 13
- 3.3 Prescan example . . . 14

4 Conclusions 16

List of Figures

1.1 The novel cognitive architecture in SymAware. 6
1.2 The main (technical) work packages of SymAware. 6

2.1 SymAware git repository. 8
2.2 SymAware technical documentation. 9

3.1 Result of the simple example with six agents. 12
3.2 Result of the communication example. 13
3.3 Result of the prescan example. 14

Acronyms

CI/CD Continuous Integration/Continuous Deployment.

EIC European Innovation Council.

MAS Multi-Agent System.

WP Work Package.

1. Introduction

This report is the deliverable for Task 7.8 in the second year of the SymAware project. It is based on the following research contributions:

- Ernesto Casablanca, Zengjie Zhang, Gregorio Marchesini, Sofie Haesaert, Dimos V. Dimarogonas, and Sadegh Soudjani, "SymAware: A Software Development Framework for Trustworthy Multi-Agent Systems with Situational Awareness" preparing for the ICRA2025 conference [submitted] [1]. (Collaborators: MPI-SWS, KTH, TUE)

The paper is accompanied by a video [Link] available on the "SymAware Project" YouTube channel [Link] showcasing the case studies we presented.

These works focus on advanced techniques in distributionally robust control and risk-based control frameworks, with a particular emphasis on addressing Signal Temporal Logic (STL) specifications under conditions of uncertainty and limited information. Conducted as collaborative efforts between MPI-SWS and KTH, with MPI-SWS leading the research, these studies contribute to the objectives of the SymAware project. They advance both the theoretical foundations and practical methods for task planning and control in complex, multi-agent environments where available information is restricted.

1.1 SymAware: High-level Description of the Project

SymAware addresses the fundamental need for a new conceptual framework for awareness in Multi-Agent Systems (MASs) that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans. The goal of SymAware is to provide a comprehensive framework for situational awareness to support sustainable autonomy via agents that actively perceive risks and collaborate with other robots and humans to improve their awareness and understanding while fulfilling complex and dynamically changing tasks. The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions. These objectives will be achieved in SymAware through

1. Logical characterization of awareness using symbolic methods,
2. Quantifying the symbolic reasoning for awareness with spatial and temporal ingredients for decision making,

3. Risk awareness via quantified logic,
4. Quantifying and communicating knowledge awareness,
5. Demonstrating awareness engineering in aviation and automotive use cases, and
6. Identifying requirements for ethical and trustworthy awareness in human-agent interaction.

The objectives of SymAware address the "Awareness Inside" Challenge of the European Innovation Council (EIC) by extending and formalizing human-based models of situational awareness and by providing a novel conceptual situational awareness framework for MASs that encompasses logical characterisation and integrative formal reasoning of interdependent awareness dimensions including knowledge, spatiotemporal, risk and social dimensions. This will support transitioning to safe mixed operation of autonomous agents and humans.

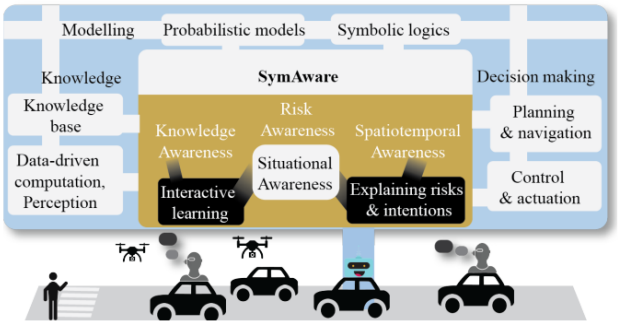

Figure 1.1: The novel cognitive architecture in SymAware.

Figure 1.1 shows a schematic view of the SymAware new cognitive awareness architecture. Together the objectives of this proposal will achieve the depicted situational awareness for multiagent settings in which the situational awareness is related to the different awareness dimensions and to contemporary methods for probabilistic modelling, perception, estimation, and (data-driven) computation.

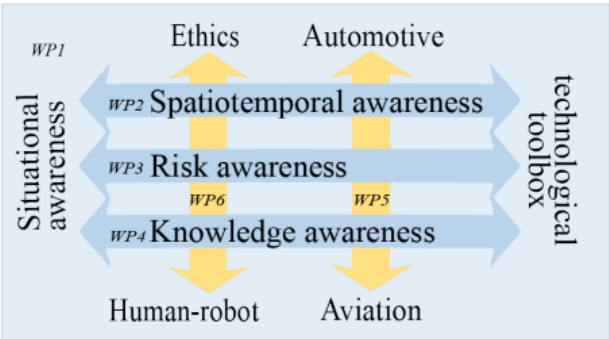

Figure 1.2: The main (technical) work packages of SymAware.

Figure 1.2 shows the main technical work packages of SymAware. The comprehensive framework of situational awareness and its logical characterisation (Objective 1) will be developed in Work

Package (WP)1. The spatiotemporal awareness of Objective 2 will be studied in WP2. Risk and knowledge awareness (Objectives 3 and 4) will be studied in WP3 and WP4, respectively. The automotive and aviation use cases (Objective 5) will be developed in WP5. Finally, the human-robot interaction and the ethical requirements (Objective 6) will be studied in WP6.

1.2 Purpose and Contents of This Report

This report is the third deliverable of WP7 on dissemination and exploitation and specifically on Task 7.8 about software tools on public Git repository (second release).

The report is organized as follows: Chapter 2 shows how the framework has been made available and open-source (i.e. the repository organization and the installation process), Chapter 3 shows some examples of the software in action.

2. Git repository

2.1 Repository

The code is completely open-source, being hosted on MPI's GitLab [1] (Fig. 2.1). Linting and

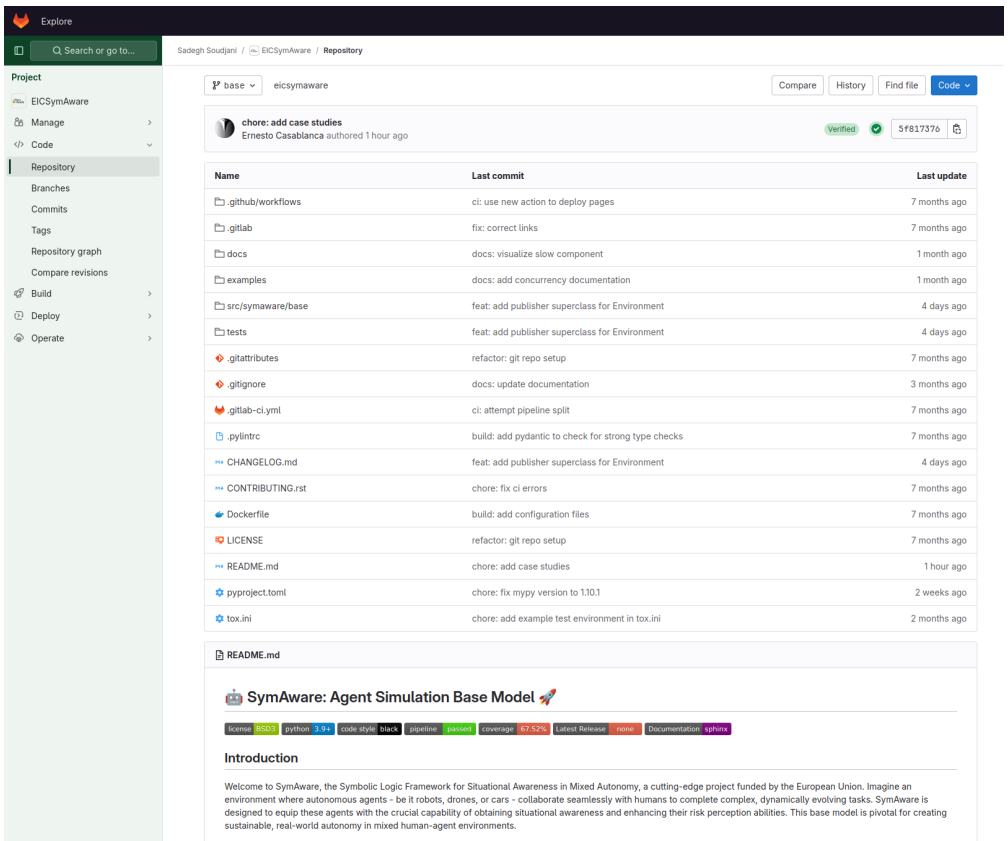

Figure 2.1: SymAware git repository.

unit-testing are performed at each commit on the branch using GitLab actions, running on all versions of Python ≥ 3.9. The resulting test coverage percentage is displayed on the README file. Great effort has been put into ensuring the correct use of all the type hinting features of Python,

[1] GitLab repository: https://gitlab.mpi-sws.org/sadegh/eicsymaware/-/tree/base?ref_type=heads

which are checked by the *mypy* static type checker. The technical documentation (Fig. 2.2) is automatically obtained from the source code's docstrings and integrated through Markdown files, leveraging Sphinx's many extensions. It is hosted and publicly available on the project's GitLab page [2] and it is updated along with the software.

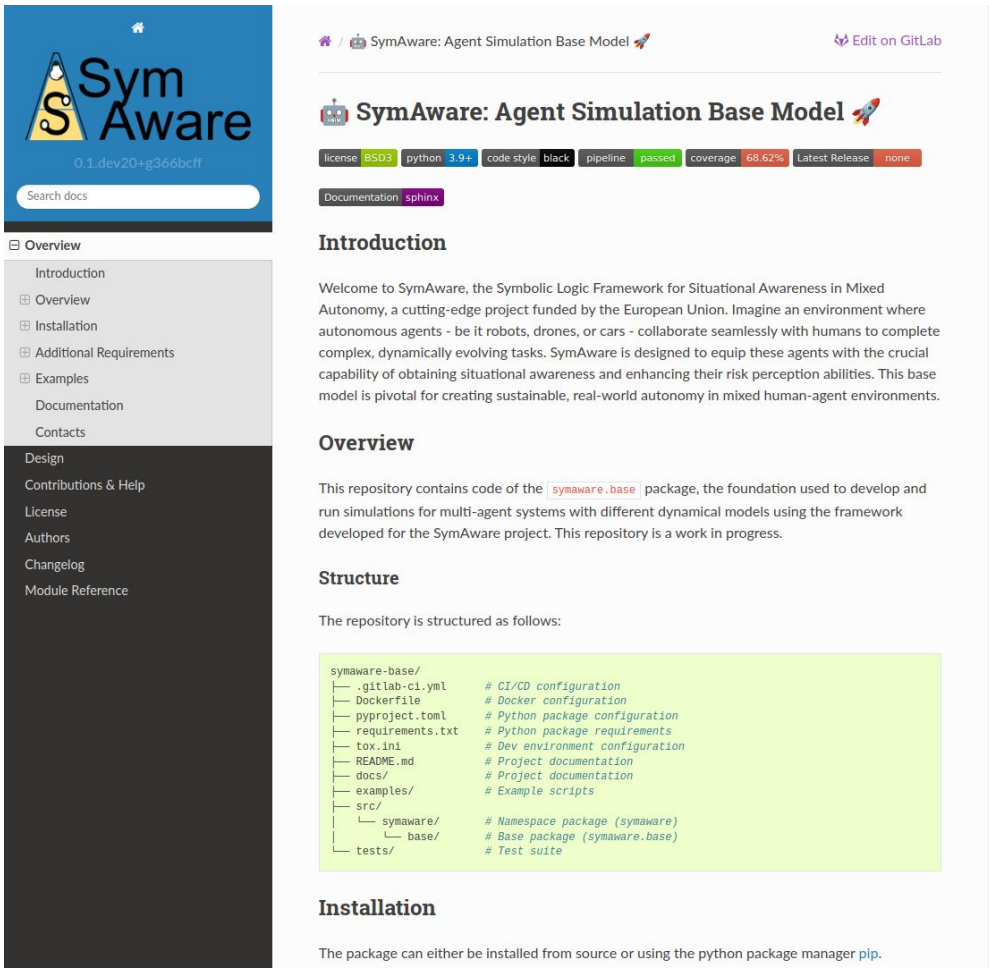

Figure 2.2: SymAware technical documentation.

2.2 Installation

The Continuous Integration/Continuous Deployment (CI/CD) pipeline is set up to automatically build and publish the packages to the GitLab package registry whenever a new tag is pushed to the repository. There are currently four packages available: *symaware-base*, *symaware-pybullet*, *symaware-pymunk* and *symaware-prescan*.

They can be installed utilising pip, which takes care of all the transitive dependencies. Running the following command will install the base package:

[2] Documentation: https://sadegh.pages.mpi-sws.org/eicsymaware/index.html

```
$ pip install symaware-base --index-url \
    "https://gitlab.mpi-sws.org/api/v4/projects/2668/packages/pypi/simple"
```

To use a specific simulator, i.e. PyBullet, install the corresponding package.

```
$ pip install symaware-pybullet --index-url \
    "https://gitlab.mpi-sws.org/api/v4/projects/2668/packages/pypi/simple"
```

The *symaware-prescan* package is an exception to the rule. Prescan requires a license to use, therefore package must be cloned from the corresponding branch of the GitLab repository and installed following the instructions available in the README file, after having made sure all the dependencies are met.

3. Examples

The repository contains an `examples` folder with a few scripts showcasing the use of the library in some common scenarios. They only require the installation of the base package and the specific simulator sub-package to run.

3.1 Simple example

The simple example showcases the core functionalities of the SymAware framework (Fig. 3.1). It uses pybullet as the underlying physics engine, and the simulation is run synchronously. The steps are as follows:

1. Define a custom controller by extending the base controller.
2. Instantiate the environment.
3. Add non-agent entities to the environment.
4. Instantiate the agents.
5. Add the agents to the environment.
6. Assemble the agents with their components.
7. Initialise all agents, determining their initial state and knowledge.
8. Add the agents to the agent coordinator.
9. Run the simulation.

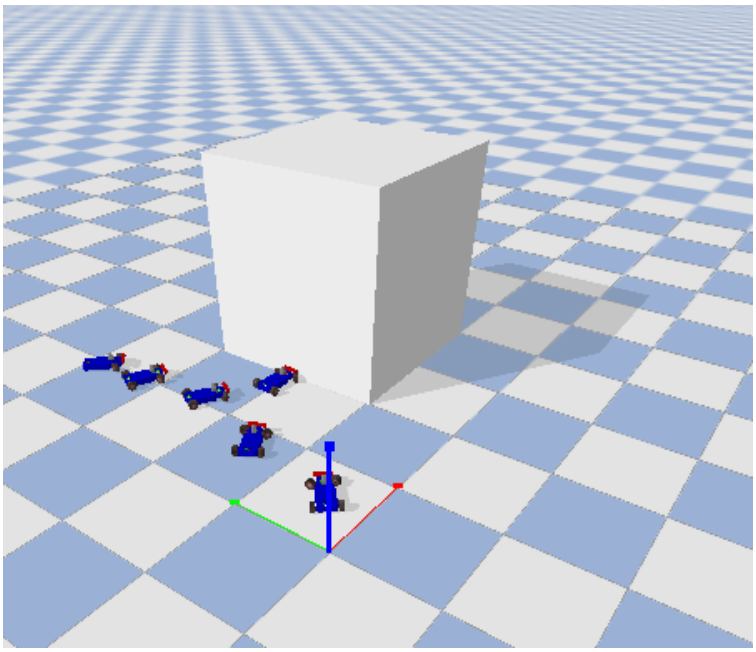

Figure 3.1: Result of the simple example with six agents. It uses the pybullet physics engine.

The source code looks something like this:

```
class MyController(Controller):
    ...

env = Environment()
env.add_entities((BoxEntity(), SphereEntity()))

agent_coordinator = AgentCoordinator[MyKnowledgeDatabase](env)

for i in range(NUM_AGENTS):
    agent = Agent[MyKnowledgeDatabase](i, RacecarEntity(i, model=RacecarModel(i)))
    env.add_agents(agent)
    agent.add_components(
        MyController(agent.id),
        DefaultPerceptionSystem(agent.id),
    )
    awareness_vector = AwarenessVector(agent.id, np.zeros(7))
    knowledge_database = MyKnowledgeDatabase(goal_pos=np.array([2, 2]))
    agent.initialise_agent(awareness_vector, {agent.id: knowledge_database})

    agent_coordinator.add_agents(agent)

agent_coordinator.run(TIME_STEP)
```

3.2 Communication example

The communication example showcases the extensibility of the communication component. It allows two agents running in two completely separated scripts (even the underlying simulator is different) to communicate with each other using a custom message, coordinating their actions (Fig. 3.2).

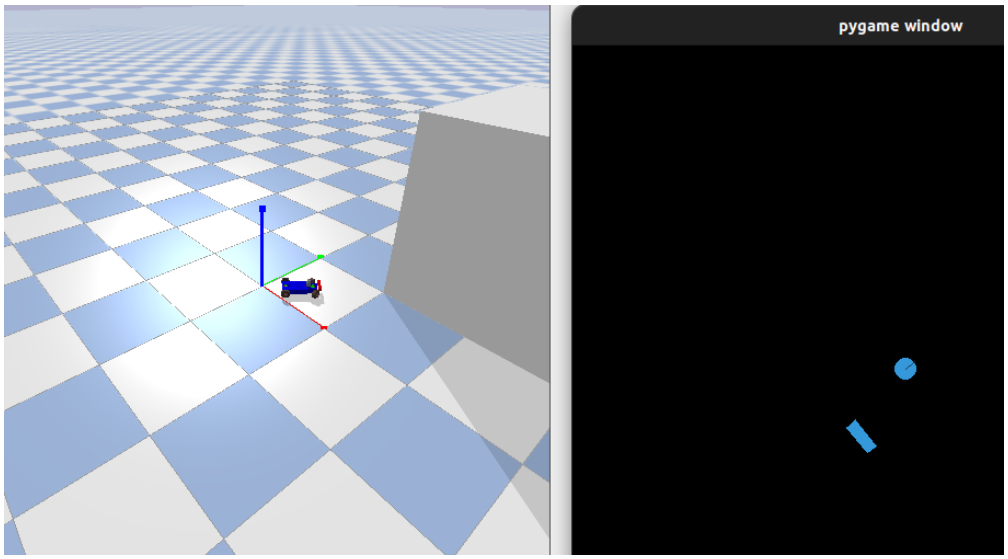

Figure 3.2: Result of the communication example. Two scripts are running at the same time. The one on the left utilises pybullet, while the one on the right pymunk. Despite being completely independent, they are able to coordinate their actions.

We need to extend the base communication components to support TCP communication.

```
class MyCommunicationSender(CommunicationSender[EventAsyncLoopLock]):
    ...

    async def _async_send_communication_through_channel(self, message):
        try:
            _, writer = await asyncio.open_connection(self._dest_ip, self._dest_port)
        except ConnectionRefusedError:
            return
        data = pickle.dumps(message)
        writer.write(data)
        await writer.drain()
        writer.close()
        await writer.wait_closed()

class MyCommunicationReceiver(CommunicationReceiver[DefaultAsyncLoopLock]):
    ...

    async def _async_receive_communication_from_channel(self):
```

```
        try:
            self._get_message_task = asyncio.create_task(self._message_queue.get())
            message = await self._get_message_task
            return (message,)
        except asyncio.CancelledError:
            return tuple()
```

3.3 Prescan example

The prescan example showcases the use of the sub-package *symaware-prescan* (Fig. 3.3).

Figure 3.3: Result of the prescan example.

We start by importing a scenario from the ones found among the simulator's demos.

```
env = Environment(filename="Demo_AmesimPreconfiguredDynamics.pb)
```

Then, we modify it to our needs, connecting to the existing entities, moving them around or deleting them and adding new ones. Entities can also be connected to sensors they will use to perceive the environment.

```
env.add_entities(
    DeerEntity(position=np.array([0, 4, 0]), orientation=np.array((0, 0, -90)))
)
target_entity = ExistingEntity(
```

```
    id=-1,
    object_name="BMW_X5_SUV_1",
    model=TrackModel(-1, existing=True)
)
env.add_entities(target_entity)
```

Lastly, we add our agent, and hook it up with our custom controller.

```
agent_entity = ExistingEntity(
    id=1,
    object_name="Audi_A8_Sedan_1",
    sensors=(AirSensor(existing=True), LmsSensor(existing=True)),
    model=AmesimDynamicalModel(1, is_flat_ground=False, existing=True),
)
agent = Agent[MyKnowledgeDatabase](1, agent_entity)
env.add_agents(agent)

controller = MyController(agent.id)
agent.add_components(controller, DefaultPerceptionSystem(agent.id, env))
# This forces the controller to be called at each simulation step
# even if the simulation is asynchronous
env.set_on_pre_step(controller.on_pre_step)
```

4. Conclusions

This report shows the open-source nature of the SymAware framework, its source code publicly available on the MPI's git repository and the simple procedure necessary to install it as a Python package. The example scripts provided can guide new users in approaching the framework for the first time.

The third release of the tool will be published on the official Python Package Index, Pypi, and will include the integration of all the partners' case works within the framework, as well as bug fixes and additional examples to cover more use cases. To ensure compartmentalization, as the number of sub-packages grows, so will the number of repositories. They will still be reachable from the original one by being included as submodules.

Bibliography

[1] E. Casablanca, Z. Zhang, G. Marchesini, S. Haesaert, D. V. Dimarogonas, and S. Soudjani, "Symaware: A software development framework for trustworthy multi-agent systems with situational awareness," 2024. [Online]. Available: https://arxiv.org/abs/2409.14833